

Report - Version 4: Mini-Batch Neural Network

Dataset	Task	Architecture	Test acc.
Fashion-MNIST	10 classes	784 -> 64 -> 10	87.28%

Introduction

Version 4 keeps the same 784 -> 64 -> 10 neural network introduced in Version 3, but changes how it is trained. Instead of using the entire training set for one update, the data is divided into mini-batches and the parameters are updated after each batch.

Goal of Version 4

The main goal is to understand mini-batch gradient descent, why repeated smaller updates can improve learning, and why the training data is shuffled at the beginning of every epoch.

Data and Preparation

The project uses the Fashion-MNIST CSV dataset with 60,000 labeled training images and 10,000 labeled test images. Each 28 x 28 grayscale image is flattened into 784 pixel values. All 10 classes are used. Pixel values are normalized from 0-255 to 0-1, and labels are converted to one-hot vectors.

Architecture

The network architecture is intentionally unchanged from Version 3:

784 pixels -> 64 hidden neurons -> ReLU -> 10 output scores -> softmax -> argmax

Mini-Batch Training

Training uses mini-batches of 64 examples. For each batch, the network performs a forward pass, computes the cross-entropy loss, runs backpropagation, and immediately updates all weights and biases.

Shuffling

At the beginning of every epoch, the training examples are randomly reordered. X and y are shuffled with the same indices so that every image remains paired with its correct label.

Forward Pass and Backpropagation

Each mini-batch passes through the hidden layer, ReLU, the output layer, and softmax. The output error is propagated backward through the second layer and then through the ReLU derivative to compute gradients for w_2 , b_2 , w_1 , and b_1 .

Loss and Numerical Safety

Multiclass cross-entropy measures the prediction error. Before applying the logarithm, predicted probabilities are clipped to a very small positive minimum value. This prevents $\log(0)$ without changing the practical behavior of the model.

Training

Training uses learning rate 0.1, 200 fixed epochs, and batch size 64. The model therefore updates its parameters many times during each epoch instead of only once. The reported training loss is the mean of the mini-batch losses from the final epoch.

Training Result

After 200 epochs, the mean loss of the final epoch is 0.0695 and training accuracy is 95.75%.

Evaluation

The full training and test sets are evaluated with the same two-layer forward pass. The class with the highest softmax probability is selected with argmax . Test accuracy is 87.28%.

Results

Metric	Value
Final epoch mean loss	0.0695
Training accuracy	95.75%
Test accuracy	87.28%

What Version 4 Teaches

Version 4 shows that the training strategy can matter as much as the architecture. The network is unchanged, but mini-batch updates and shuffling make learning much more effective than the previous full-batch configuration.

Current Performance

With this educational configuration, the network reaches 95.75% training accuracy and 87.28% test accuracy. The improvement in test accuracy shows that mini-batch training is a much stronger optimization strategy for this model.

Generalization Gap

Training accuracy is noticeably higher than test accuracy. This gap indicates that the model fits the training data better than unseen data, so later versions can explore architecture or regularization choices that improve generalization.

Limitations

Version 4 still uses one hidden layer, a fixed learning rate, plain gradient descent, and no regularization or advanced optimizer. The batch size is fixed at 64 and the architecture remains deliberately small and easy to inspect.

Conclusion

Version 4 successfully introduces mini-batch gradient descent, epoch-level shuffling, safer cross-entropy evaluation, and repeated parameter updates while preserving the same transparent two-layer network. It creates the training foundation needed for the deeper network planned in Version 5.